Disjoint Set Union in Competitive Programming

The Dynamic Connectivity Problem

Defining the Problem

- We start with N elements, initially partitioned into N distinct, isolated sets.
- We must efficiently support two core operations:
  - `find(x)`: Identify the unique representative (the "root") of the set containing x.
  - `union(x, y)`: Merge the sets containing x and y into a single set.
- The network changes dynamically; queries and updates are interleaved.

Why Standard Traversals Fail

- → **The Graph Traversal Approach:** Run DFS or BFS after every edge addition to determine if two nodes are connected.
- → **The Bottleneck:** A single traversal requires $\mathcal{O}(N + M)$ time.
- → For Q dynamic queries, the worst-case time complexity degrades to $\mathcal{O}(Q \times (N + M))$.
- → In standard competitive programming bounds ($N, M, Q \approx 2 \cdot 10^5$), this approach guarantees a Time Limit Exceeded (TLE) verdict.

Enter Disjoint Set Union (DSU)

- → DSU provides a specialized data structure strictly optimized for the dynamic connectivity problem.
- → **Target Time Complexity:** Nearly $\mathcal{O}(1)$ amortized time per query.
- → **Target Space Complexity:** $\mathcal{O}(N)$ to maintain the parent relationships.
- → **Trade-off:** Standard DSU cannot "un-merge" sets or handle edge deletions efficiently. It is purely an incremental data structure.

Naive Implementation & Its Bottlenecks

Tree Representation of Sets

- → Instead of storing sets as flat arrays, we represent them as **directed trees**.
- → Every node points to a parent node.
- → The **root** of the tree (a node that points to itself) serves as the unique representative for that entire set.
- → `find(x)`: Traverse up the parent pointers until reaching the root ($parent[x] == x$).
- → `union(x, y)`: Find the roots of x and y, then make one root the child of the other.

Naive C++ Implementation

```
void make_set(int v) {
    parent[v] = v;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b)
        parent[b] = a;
}
```

Complexity Analysis

- → **Space Complexity:** $O(N)$ to store the `parent` array.
- → **Time Complexity (Best Case):** $O(1)$ if the tree is completely flat.
- → **Time Complexity (Worst Case):** $O(N)$ per operation.
- → **The Bottleneck:** If we sequentially `union(1, 2)`, `union(2, 3)`, ..., `union(N-1, N)`, the tree degrades into a straight line (a linked list).
- → A subsequent `find(1)` will require traversing all N nodes.

Pro-Tip: Malicious Test Cases

- → **Never use naive DSU in CP** unless N is trivially small (e.g., $N \leq 100$).
- → Problem setters actively design test cases (often called "star graphs" or "deep lines") specifically to exploit the $O(N)$ worst-case of a naive DSU.
- → Submitting naive DSU on standard constraints ($N = 10^5$) will consistently yield a Time Limit Exceeded (TLE) verdict.

Core Optimizations

Path Compression: The "Find" Optimization

- → **The Goal:** If we spend time traversing a long path to find the root, we should ensure we never pay that cost again for the same nodes.
- → **The Mechanism:** During the recursive return phase of `find(x)`, reassign the parent of every visited node to point directly to the root.
- → This aggressively "flattens" the tree structure.
- → **Complexity Impact:** Reduces the amortized time complexity of `find` to $\mathcal{O}(\log N)$.

Implementing Path Compression

```
int find(int x) {
    if (x == parent[x]) {
        return x;
    }
    // Path compression: update parent on the way back up
    return parent[x] = find(parent[x]);
}
```

→ **Caveat:** This requires the underlying parent array to be mutable during a conceptually "read-only" operation.

Union by Size: The "Union" Optimization

- → **The Goal:** Prevent the tree from becoming deep in the first place when merging two distinct sets.
- → **The Mechanism:** Maintain the size (number of elements) of each set. Always attach the root of the *smaller* tree as a child of the root of the *larger* tree.
- → **Mathematical Guarantee:** The depth of any node can only increase if its tree is merged into a tree of equal or greater size. Thus, the maximum depth is strictly bounded by $\log_2(N)$.
- → **Complexity Impact:** Operations take strictly $\mathcal{O}(\log N)$ time, even without path compression.

Implementing Union by Size

```
bool unite(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);
    if (rootX == rootY) return false;

    // Ensure rootX represents the larger tree
    if (size[rootX] < size[rootY]) {
        swap(rootX, rootY);
    }

    parent[rootY] = rootX;
    size[rootX] += size[rootY];
    return true;
}
```

Alternative Heuristics: Randomized Linking

- → **The Concept:** If maintaining a `size` or `rank` array introduces unwanted overhead, randomized heuristics offer a probabilistic alternative.
- → **Coin-Flip Linking:** When merging `rootX` and `rootY`, generate a random boolean to decide which becomes the parent.
- → **Why it works:** It explicitly breaks deterministic, malicious test cases designed by problem setters to force the worst-case $\mathcal{O}(N)$ linked-list structure.
- → **Use Case:** Highly effective in tight memory limits where saving the $\mathcal{O}(N)$ space of the size array is required, though Union by Size remains the standard for general competitive programming.

Complexity Analysis

The Ackermann Function

- → To understand the complexity of an optimized DSU, we must first look at the Ackermann function, $A(m, n)$.
- → It is a computable function that is strictly not primitive recursive.
- → **Key Characteristic:** It grows at an extraordinarily rapid rate.
- → For context, $A(4, 2)$ yields a number with $19,729$ decimal digits. $A(4, 3)$ is vastly larger than the estimated number of atoms in the observable universe.

The Inverse Ackermann Function $\alpha(N)$

- → The inverse Ackermann function, denoted as $\alpha(N)$, essentially asks: "How many times must we apply the inverse operation to reach a base case?"
- → Because $A(m, n)$ grows unfathomably fast, its inverse $\alpha(N)$ grows unfathomably slowly.
- → For any physically meaningful value of N, $\alpha(N) \leq 4$.
- → Specifically, $\alpha(N) \leq 4$ for all $N < 2^{2^{2^{65536}}} - 3$.

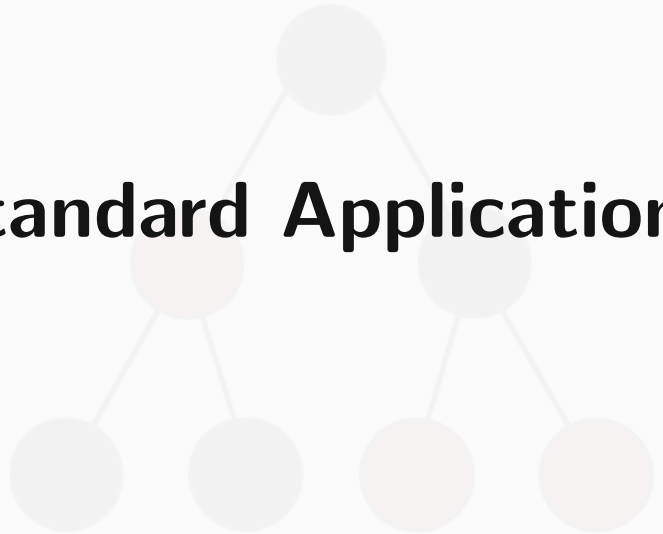
Amortized Complexity of DSU

- → **The Theorem:** When combining **Path Compression** and **Union by Size/Rank**, a sequence of M operations on N elements takes $\mathcal{O}(M\alpha(N))$ time.
- → The amortized time complexity per operation is therefore strictly bounded by $\mathcal{O}(\alpha(N))$.
- → **Mathematical Distinction:** While $\alpha(N) \leq 4$ in practice, $\mathcal{O}(N\alpha(N))$ is *not* mathematically equivalent to strictly linear $\mathcal{O}(N)$ time. It is technically superlinear, albeit by a microscopic margin.

Practical Implications in Competitive Programming

- → **Is it $\mathcal{O}(1)$?** In software engineering practice, yes. In asymptotic theory, no.
- → **Performance Bottlenecks:** You will never receive a Time Limit Exceeded (TLE) due to the $\alpha(N)$ scaling factor.
- → If an optimized DSU solution TLEs, the issue is typically:
  - → High constant factors (e.g. excessive cache misses, or failing to use path compression and suffering deep recursion).
  - → Overusing heavy data structures (like `std::map`) alongside the DSU operations.
  - → Other algorithmic phases dominating the runtime (e.g., sorting edges in Kruskal's algorithm takes $\mathcal{O}(M \log M)$).

Standard Applications



Application 1: Dynamic Connected Components

- **The Problem:** Given a graph where edges are added sequentially, efficiently query the number of connected components at any time.
- **The DSU Approach:**
  - Initially, with N isolated vertices, there are exactly N components.
  - Every time an edge successfully connects two *previously disconnected* vertices (i.e., `union(u, v)` merges two distinct roots), the total number of components decreases by exactly 1.
- **Formula:** Current Components $= N -$ Successful Unions.

Tracking Components in C++

```
int components = n;

bool unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return false;

    if (size[rx] < size[ry]) swap(rx, ry);
    parent[ry] = rx;
    size[rx] += size[ry];

    components--; // Decrement on successful merge
    return true;
}
```

Application 2: Kruskal's Algorithm (MST)

- → **The Problem:** Find a subset of edges that connects all vertices in a weighted graph with the minimum possible total edge weight.
- → **The Algorithm:**
  1. Sort all edges in non-decreasing order of weight.
  2. Iterate through the sorted edges.
  3. For each edge (u, v), use DSU to check if u and v are in the same set.
  4. If they are not (`find(u) != find(v)`), add the edge to the MST and `union(u, v)`.
- → **Why it works:** Processing edges in ascending order guarantees that we connect components using the cheapest possible edge, while the DSU inherently prevents cycle formation.

Implementing Kruskal's

```
// edges: vector of pair<weight, pair<u, v>>
sort(edges.begin(), edges.end());

long long mst_weight = 0;
int edges_used = 0;

for (auto& edge : edges) {
    if (dsu.unite(edge.second.first, edge.second.second)) {
        mst_weight += edge.first;
        edges_used++;
    }
    // Optimization:
    // MST is complete when N-1 edges are added
    if (edges_used == N - 1) break;
}
```

Problem Walkthrough: Cycle Detection

- → **Classic Problem Formulation:** "Redundant Connection"
- → **Statement:** You are given a tree with N nodes and N edges (meaning exactly one extra edge has been added, creating a single cycle). Find an edge that can be removed to restore the strict tree structure.
- → **The DSU Solution:**
  - → Initialize a DSU with N elements.
  - → Iterate through the given edges sequentially.
  - → For each edge (u, v), attempt to perform `unite(u, v)`.
  - → If `unite(u, v)` returns `false` (meaning u and v already share the same root), then adding this specific edge creates the cycle. This is the redundant connection.
- → **Complexity:** $\mathcal{O}(N\alpha(N))$